

The lua-tikz3dtools Package

The lua-tikz3dtools Package, Version 2.0.0

<https://github.com/Pseudonym321/TikZ-Animations/tree/master1/TikZ/lua-tikz3dtools>

Jasper Nice

September 4, 2025

This work is licensed under the LaTeX Project Public License, version 1.3c or later.
—Jasper Nice

Dedicated to those who, like me, never found the tools they needed—and chose to build them.

“I long to accomplish great and noble tasks but it is my duty to accomplish small tasks as though they were great and noble.” —Sylvia Fedoruk

Contents

Preface	xi
Acknowledgements	xiii
1 Problem Statement	1
1.1 Background and Context	1
1.2 The Core Problem Being Addressed	1
1.3 Scope and Boundaries of the Problem	1
1.4 Importance of the Issue	2
1.5 Current Limitations and Gaps	2
1.6 Objectives and Goals	3
1.7 Intended Audience	3
1.8 Summary of the Problem Statement	3
2 Literature Review	5
2.1 Introduction to the Literature Review	5
2.2 Historical Background and Evolution of the Field	5
2.3 Existing Approaches in Practice and Academia	6
2.4 Existing Approaches in Practice and Academia	6
2.5 Comparative Analysis of Approaches	6
2.6 Strengths and Contributions of Current Work	7
2.7 Limitations and Open Challenges in Current Methods	7
2.8 Emerging Trends and Future Directions	7
2.9 Proposed Approach and Its Advantages Over Existing Work	8
2.10 Summary of the Literature Review	8
3 Methodology	9
3.1 Introduction to the Methodology	9
3.2 Research Design and Overall Approach	10
3.3 Detailed Description of the Proposed Approach	10
3.4 Rationale and Development Process of the Approach	10
3.5 Data, Tools, and Resources Used	10
3.6 Implementation Details	10
3.7 Validation Strategy	10

3.8	Challenges Encountered and Solutions Implemented	10
3.9	Limitations of the Current Methodology	10
3.10	Remaining Challenges and Directions for Future Work	10
3.11	Summary of the Methodology	10
4	Validation	11
4.1	Introduction to Validation	11
4.2	Internal Evaluation	11
4.2.1	Performance Assessment	11
4.2.2	Pedagogical Effectiveness of Visualizations Compared to Existing Methods	11
4.2.3	Robustness and Reliability Testing	11
4.3	Comparative Evaluation Against Benchmarks	11
4.4	Feedback from Domain Experts	11
4.5	User Study or Field Evaluation (if applicable)	11
4.6	Limitations of the Validation Process	11
4.7	Summary of Validation Results	11
5	Results and Analysis	13
5.1	Introduction to Results and Analysis	13
5.2	System Achievements and Capabilities	13
5.2.1	Functional Capabilities	13
5.2.2	Performance Metrics	13
5.2.3	Comparative Advantages Over Existing Methods	13
5.2.4	Limitations Observed in Practice	13
5.3	Validation Results and Interpretation	13
5.3.1	Internal Evaluation Outcomes	13
	Performance Assessment	13
	Pedagogical Effectiveness of Visualizations	13
5.3.2	Expert Feedback Analysis	13
5.3.3	User Study or Broader Evaluation (if applicable)	13
5.4	Synthesis and Interpretation of Results	13
5.5	Implications of Findings	13
5.6	Summary of Results and Analysis	13

List of Tables

List of Figures

Preface

Purpose of This Work

I undertook this work because I was not satisfied with the existing tools for illustrating scenes with 3D parametric objects in $\text{\LaTeX}$, such as surfaces. From the perspective of computer graphics as a whole, the ability to occlusion-order non-intersecting and non-cyclically overlapping 0–2-dimensional affine simplices in 3D—together with the systematic partitioning of simplices to eliminate intersections and cyclic overlaps—is a foundational advance. While my own motivation arose from the needs of a $\text{\LaTeX}$ illustrator, I recognized that this framework represents a paradigm shift in mathematics illustration and, more broadly, a landmark contribution to computer graphics. My goals were therefore twofold: to create the kind of illustration tool I had long envisioned, and to establish priority in implementing this transformative idea.

I have developed a systematic method for occlusion ordering 0–2-dimensional affine simplices in three dimensions. In addition, I have implemented a framework for eliminating intersections through partitioning, with the resolution of cyclic overlaps planned for the next version of the software. This work matters because it provides the only known approach for generating scenes with parametric objects that both intersect coherently and are occlusion-sorted robustly. In short, this is a rigorous and comprehensive software framework for handling intersections and occlusion ordering of affine simplices in three dimensions.

Context and Background

This book documents the $\text{\LaTeX}$ package `lua-tikz3dtools`, which implements a groundbreaking algorithm for 3D illustrations composed of 0–2-dimensional affine simplices. Although the software is designed for creating illustrations in $\text{\LaTeX}$, the underlying method represents a significant advance for the broader field of computer graphics.

Acknowledgments of Guidance and Support

I am deeply grateful to the $\text{\TeX}$ Stack Exchange community, whose many contributions and generous assistance have been invaluable in helping me develop my skills as an

illustrator. I also acknowledge the Math Stack Exchange community for several insights that proved helpful along the way. This work would not have been possible without the years of learning made available by my peers on T_EX Stack Exchange, nor without the foundational training in programming and linear algebra that I received from my professors at my local university.

Author's Contribution and Perspective

I am the principal designer of nearly all of `lua-tikz3dtools`, with a few specific components assisted by ChatGPT. My two primary original contributions are the occlusion-ordering algorithm and the simplex-partitioning algorithm. The portion produced with ChatGPT's assistance concerns the recursive traversal of segments that applies my algorithms. Apart from this limited assistance, the work was carried out independently, with valuable input and feedback from the T_EX Stack Exchange community.

Structure of the Document

This book is organized into five chapters. Chapter 1 presents the problem statement, defining and framing the central challenge. Chapter 2 reviews the relevant literature. Chapter 3 describes the methodology, including the algorithms developed. Chapter 4 provides validation of the approach, and Chapter 5 presents the results and analysis.

Acknowledgements

This work was typeset using $\text{\TeX}$, the typesetting system created by Donald E. Knuth, along with various extensions and packages developed by the $\text{\TeX}$ community. I am grateful to the vibrant $\text{\TeX}$ Stack Exchange community for their ongoing support and resources. For those interested, my contributions can be found at [Jasper \[Jas\]](#).

I also acknowledge the assistance of ChatGPT, which contributed to parts of the code and to the refinement of this documentation.

Jasper Nice

Chapter 1

Problem Statement

1.1 Background and Context

The field to which `lua-tikz3dtools` belongs is inherently multidisciplinary, combining elements of programming and mathematics, especially linear algebra. In my view, the current state of practice remains fragmented. Much of the existing literature focuses on isolated components of the broader problem; for example, publications may address tessellation of surfaces or the clipping of line segments by triangles, but seldom attempt a unified framework. I believe I am the first to orchestrate linear algebra and programming into a coherent synthesis that enables parametric object generation tessellated by affine simplices (points, line segments, and triangles). These simplices are then systematically clipped against one another using a novel algorithm that eliminates intersections. A second novel algorithm, also of my design, performs occlusion sorting of the resulting simplices via a transitive partial order.

This represents a genuinely groundbreaking achievement, with applications in computer graphics, computational geometry, and mathematical illustration.

1.2 The Core Problem Being Addressed

The central problem addressed by `lua-tikz3dtools` is the illustration of 3D scenes composed of 0–2-dimensional affine simplices. Specifically, the package focuses on the representation of parametric objects tessellated into points, line segments, and triangles, including those that intersect. The primary outstanding challenge is the elimination of cyclic overlaps, which will be addressed in the next version of the software.

1.3 Scope and Boundaries of the Problem

The scope of the problem—excluding cyclic overlaps—is twofold: clipping intersecting simplices and partially transitive sorting of the resulting non-intersecting simplices.

This version of the package assumes that cyclic overlaps do not occur. It is designed for 3D diagrams composed of tessellated parametric objects, favoring exact partitioning over alternative methods to handle sparse objects without introducing occlusion errors.

1.4 Importance of the Issue

Until now, illustrators have often relied on high sampling to visually resolve occlusion errors arising from intersecting objects. Furthermore, to my knowledge, no software exists that explicitly sorts triangles correctly, let alone lower-dimensional affine simplices. Most existing approaches either produce improper sorting or rely on black-box methods. Solving this problem without using a z-buffer or ray tracing constitutes a significant contribution to both the theoretical and practical aspects of computer graphics, particularly in the domain of mathematical illustration. Based on years of studying 3D graphics, I can assert that this algorithm addresses a fundamental challenge for the future of the field.

From a theoretical standpoint, the method can illustrate 3D scenes composed of points, line segments, and triangles, demonstrating that a scene of triangles can be partitioned so as to allow a partial transitive ordering, resulting in correct visual occlusion.

This work represents a major theoretical advance in the conceptualization of 3D illustrations. Its synthesis of elementary linear algebra with computational methods is unique and represents a significant step beyond the current state of the art.

1.5 Current Limitations and Gaps

Previous approaches largely neglect intersecting simplices and often produce occlusion errors. In fact, many software packages do not even triangulate surfaces correctly. For example, drawing a Klein bottle in Mathematica, Asymptote, or pgfplots results in intersections that do not occlude properly. My approach resolves this issue systematically.

Not only does it eliminate intersections, but it also applies rigorous affine linear algebra for correct occlusion sorting. These gaps in existing methods persist largely because no one has previously pursued a comprehensive study of 3D mathematical illustration with this level of coherence. While many practitioners achieve impressive results, none combine triangulation, clipping, and occlusion sorting in a fully integrated framework. Unlike approaches that rely on splines or high sampling, my package ensures accurate visualization of sparse geometries.

High sampling is computationally expensive and often insufficient, since the human eye can detect very small errors. Achieving the necessary resolution via sampling alone is generally impractical. My software addresses this challenge directly by first clipping simplices and then performing occlusion sorting, ensuring correct visual results efficiently.

1.6 Objectives and Goals

The objective of this documentation is to present the software I have developed, demonstrate its correctness, and provide guidance on its use. Detailed explanations are provided for each of the novel algorithms, accompanied by examples illustrating how to apply the software in practice. In addition, practical use cases are discussed, and the software is validated through tests.

1.7 Intended Audience

This documentation is intended for 3D mathematical illustrators who are already familiar with basic programming, linear algebra, and TikZ. While the package primarily supports 3D mathematical illustrations in $\LaTeX$, its underlying algorithms have implications beyond the $\LaTeX$ community. Computer graphics researchers will benefit from the first coherent synthesis of a transitive partial-order algorithm for occlusion sorting of affine simplices.

Mathematics textbook authors and illustrators will also benefit from a robust backend for occlusion handling and clipping, without relying on black-box software that produces incorrect results. Although some tools generate visually plausible output for users with limited 3D intuition, subtle errors often remain undetected. The approach presented here provides a rigorous and systematic solution, and it is likely that the software will have applications beyond these immediate use cases.

1.8 Summary of the Problem Statement

The central problem addressed by this documentation is the clipping and occlusion ordering of affine simplices. In other words, it concerns the visualization of 3D parametric scenes. This software is significant because it represents the first coherent synthesis for illustrating such scenes.

Chapter 2

Literature Review

2.1 Introduction to the Literature Review

The purpose of this chapter is to situate this work within the context of existing scholarship. The chapter is organized chronologically and reflects my personal journey along this path. Because my synthesis of the field is genuinely novel, most of the sources discussed are my own contributions, accompanied by testimony that contextualizes and motivates these developments.

2.2 Historical Background and Evolution of the Field

I began as a complete amateur, initially motivated by a desire to create mathematical textbook illustrations. Over time, I developed a particular interest in 3D illustrations, but quickly grew dissatisfied with the state of the available tools and resolved to build them myself. My first attempt involved tessellating a surface into quadrilaterals, but I soon realized that the occlusion was incorrect. At the time I was taking my first course in linear algebra, and with the help of ChatGPT [Ski25], I recognized that the dot product could be used to approximate depth-ordering by midpoint comparisons. I later refined this idea through a Mathematics Stack Exchange inquiry [httb]. By intuition, I also realized that tessellating into triangles was more coherent than quadrilaterals, which are often non-coplanar. Initially, I attempted to sort the triangles by their midpoints, but the results were still inadequately occluded.

This led me to another inquiry on occlusion ordering of non-intersecting, non-cyclically overlapping triangles [htta], where a pivotal suggestion was made in the comments: to sort only simplices whose orthogonal projections on the viewing plane overlap. This insight proved to be the breakthrough. From this seed grew the present occlusion algorithm. In the final system, simplices are first projected onto the viewing plane; when an overlap is detected, sorting is resolved by using the inverse orthogonal projection back onto both shapes. Although it took months of refinement, this ultimately yielded a coherent and general solution.

The occlusion problem also naturally arises from intersecting and cyclically overlapping simplices. My algorithm addresses the first case by completely eliminating intersections through partitioning, thereby establishing a foundation alongside the occlusion algorithm. The resolution of cyclic overlap is left for future work.

2.3 Existing Approaches in Practice and Academia

2.4 Existing Approaches in Practice and Academia

Until now, no comprehensive solution to the problem of clipping and occlusion ordering of affine simplices has existed. Existing practices in 3D mathematical illustration and computer graphics address only fragments of the problem. For example, many software systems triangulate surfaces and order the resulting triangles using primitive heuristics, but they do not perform clipping of intersecting simplices, and their occlusion methods are often approximate or incorrect.

Much of the literature in both academia and practice focuses on partial approaches—such as tessellation, polygon clipping, or depth-sorting—but not their coherent integration into a rigorous framework. This fragmentation explains why current tools, despite producing visually plausible illustrations, frequently introduce occlusion errors when tested under precise mathematical scrutiny.

The work presented here unifies these scattered threads into a single coherent system, addressing both intersection elimination and rigorous occlusion ordering. This synthesis has not been achieved previously in the literature or in practice.

2.5 Comparative Analysis of Approaches

When comparing existing approaches to the method implemented in `lua-tikz3dtools`, a fundamental distinction emerges. Existing tools and frameworks for 3D illustration—whether in academic research or practical software—typically handle only parts of the problem. Some systems focus on tessellation, others on clipping, and still others on heuristic depth-sorting. However, none integrate these processes into a coherent framework capable of systematically handling intersections and providing rigorous occlusion ordering of affine simplices.

By contrast, the algorithm presented here addresses both of these challenges directly. It introduces a systematic clipping procedure to eliminate intersections and a transitive partial-order approach to occlusion sorting. No existing approach in either the academic literature or applied software achieves this combination. In short, while prior methods provide partial or approximate solutions, the present work provides the first complete and rigorous solution to the problem of visualizing 3D parametric scenes composed of affine simplices.

2.6 Strengths and Contributions of Current Work

The present work successfully demonstrates the systematic illustration of 3D scenes composed of affine simplices. This was achieved through the use of affine linear algebra to explicitly define simplices and to traverse their spans. The span of a simplex is the set of points obtained by taking linear combinations of its basis vectors, originating from a given point.

A further breakthrough came from reducing the problem to its simplest building blocks: the intersection and occlusion of the lowest-dimensional simplices. From there, higher-dimensional simplices, such as triangles, were treated in terms of these more elementary cases. This reductionist approach provided a clear and coherent foundation for solving the problem at scale.

2.7 Limitations and Open Challenges in Current Methods

This documentation does not, in its current form, address cyclically overlapping simplices; this challenge is reserved for future work. The decision to defer this aspect was intentional, in order to secure authorship priority on the core contribution before extending the framework further.

Future versions will also introduce the ability to clip parametric objects—that is, collections of simplices belonging to the same object—by themselves. This will enable the computation of intersection sets as new, customizable parametric objects, thereby broadening the applicability of the package.

2.8 Emerging Trends and Future Directions

Recent developments in computer graphics emphasize the pursuit of photorealism, with ray tracing technologies at the forefront. These methods simulate lighting, reflection, and refraction effects at a high degree of physical accuracy. While such techniques are powerful, they are computationally intensive and primarily focused on rendering dense, visually realistic scenes rather than sparse, mathematically rigorous ones. Nevertheless, the conceptual framework of ray tracing demonstrates the importance of light-based visibility models, which may offer inspiration for extending methods of occlusion ordering in the future.

In contrast, the approach taken in this work focuses on exact algebraic partitioning and sorting of affine simplices, prioritizing mathematical rigor over visual realism. Future research may explore a hybrid direction, drawing selectively from photorealistic methods while preserving the precision needed for mathematical illustration. Such a synthesis would represent a promising new trajectory: combining the clarity and correctness demanded by pedagogy with techniques inspired by the broader field of computer graphics.

2.9 Proposed Approach and Its Advantages Over Existing Work

The limitations of existing methods are clear: they either neglect intersections altogether, rely on approximate sampling, or obscure their logic within black-box implementations. As a result, mathematical illustrators are left without a reliable way to render sparse parametric geometries in three dimensions without visible errors. This is the precise gap that motivates the present work.

The approach developed in `lua-tikz3dtools` addresses this gap by providing a systematic method for clipping intersecting simplices and establishing a rigorous transitive partial order for occlusion sorting. Unlike prior tools, this method is fully transparent in its logic, grounded in affine linear algebra, and designed specifically to ensure correctness even for sparse configurations where high-sampling techniques fail.

The principal advantage of this work is its ability to produce illustrations that are both mathematically precise and computationally efficient. By reducing the problem to its most fundamental constituents—simplices—and resolving both intersections and occlusion ordering at that level, the method achieves a coherence that existing approaches lack. This framework not only serves the immediate needs of mathematical illustration, but also establishes a foundation that can be extended to broader areas of computer graphics.

2.10 Summary of the Literature Review

The review of existing tools and practices makes one thing clear: the available methods were inadequate for producing precise 3D mathematical illustrations. They either ignored intersections, introduced occlusion errors, or relied on approximate techniques such as high sampling. Bound by the limitations of the time, I was compelled to invent my own approach. While this work does not yet address the final challenge of cyclic overlaps, it establishes a rigorous framework for clipping intersecting simplices and systematically ordering them by occlusion. The next chapter will present the methodology by which this is achieved.

Chapter 3

Methodology

3.1 Introduction to the Methodology

This chapter explains how the research was carried out and how the core algorithms were developed. The synthesis did not arise from a gradual accumulation of partial solutions, but from a decisive insight gained after years of experience with 3D illustration. That insight was to frame the entire problem in terms of affine linear algebra, which provided the clarity and rigor needed to resolve clipping and occlusion systematically.

The implementation was carried out in $\text{\LaTeX}$ and Lua, not by accident but because I am a mathematics textbook illustrator, and these are the tools I know and use daily. This choice ensured that the algorithms were embedded directly in the environment where they are most relevant, while also demonstrating their applicability beyond it. The following sections present the design, implementation, and validation of the algorithms, showing how they directly address the fundamental gaps identified in the Problem Statement and Literature Review.

- 3.2 Research Design and Overall Approach**
- 3.3 Detailed Description of the Proposed Approach**
- 3.4 Rationale and Development Process of the Approach**
- 3.5 Data, Tools, and Resources Used**
- 3.6 Implementation Details**
- 3.7 Validation Strategy**
- 3.8 Challenges Encountered and Solutions Implemented**
- 3.9 Limitations of the Current Methodology**
- 3.10 Remaining Challenges and Directions for Future Work**
- 3.11 Summary of the Methodology**

Chapter 4

Validation

4.1 Introduction to Validation

4.2 Internal Evaluation

4.2.1 Performance Assessment

4.2.2 Pedagogical Effectiveness of Visualizations Compared to Existing Methods

4.2.3 Robustness and Reliability Testing

4.3 Comparative Evaluation Against Benchmarks

4.4 Feedback from Domain Experts

4.5 User Study or Field Evaluation (if applicable)

4.6 Limitations of the Validation Process

4.7 Summary of Validation Results

Chapter 5

Results and Analysis

5.1 Introduction to Results and Analysis

5.2 System Achievements and Capabilities

5.2.1 Functional Capabilities

5.2.2 Performance Metrics

5.2.3 Comparative Advantages Over Existing Methods

5.2.4 Limitations Observed in Practice

5.3 Validation Results and Interpretation

5.3.1 Internal Evaluation Outcomes

Performance Assessment

Pedagogical Effectiveness of Visualizations

5.3.2 Expert Feedback Analysis

5.3.3 User Study or Broader Evaluation (if applicable)

5.4 Synthesis and Interpretation of Results

5.5 Implications of Findings

5.6 Summary of Results and Analysis

Bibliography

- [hta] Jasper (<https://math.stackexchange.com/users/1499599/jasper>). *Exact Triangle Sorting for Orthographic Rendering of a Triangulated Surface*. Mathematics Stack Exchange. URL:<https://math.stackexchange.com/q/5063772> (version: 2025-05-17). URL: <https://math.stackexchange.com/q/5063772>.
- [htb] P123 (<https://math.stackexchange.com/users/1499599/p123>). *How does the dot product give correct depth ordering in orthographic 3D projections?* Mathematics Stack Exchange. URL:<https://math.stackexchange.com/q/5062592> (version: 2025-05-06). URL: <https://math.stackexchange.com/q/5062592>.
- [Jas] Jasper. *TeX StackExchange user profile: Jasper*. <https://tex.stackexchange.com/users/319072/jasper>. Accessed: 2025-08-12.
- [Ski25] Skillmon. *Answer to “3d point sorting in tikz”*. Accepted answer with LaTeX3-based segment list and sorting macros. 2025. URL: <https://tex.stackexchange.com/a/734098>.